1. Problem Description

The problem involves developing a Python-based courier management system designed to register packages, assign delivery agents, track shipment statuses, and maintain complete delivery records. In real life, logistics and courier companies face immense challenges in handling large volumes of shipments, tracking package locations accurately, assigning appropriate delivery agents, and managing customer records efficiently. Without an automated system, tracking delivery performance and generating insights become manual, error-prone tasks. This system allows staff to register customers, create shipment orders, update package locations, record delivery attempts, calculate delivery charges, and safely persist customer, package, delivery, and payment records into text files while also analyzing delivery performance through reports and Matplotlib visualizations.

2. Problem Objective

> **Core Objective:** Build an automated, Python-based courier management system that successfully registers packages, assigns delivery agents, tracks shipment statuses, securely stores records in text files, handles exceptions for invalid tracking data, and visualizes delivery performance using Matplotlib to achieve an efficient, error-free logistics workflow.

3. Steps of Implementation

The implementation follows a structured modular flow, moving from core validations and records to data manipulation, persistent storage, driver execution, and final reporting:

Step / Module	Description
1. User Authentication	Restricts system access by verifying valid staff or admin credentials before allowing package and customer registration.

Step / Module	Description
2. Record Handling	Manages and processes collections of customer details, package info, and delivery records using lists and dictionaries.
3. String Manipulation	Validates and formats textual inputs such as tracking IDs, customer names, and delivery addresses to ensure data consistency.
4. File Handling	Handles persistent data storage by writing and reading customer details, package records, and delivery logs from text files while managing file I/O exceptions.
5. Main Driver Module	Acts as the entry point and execution loop connecting all operational modules together via user-friendly menus.
6. Report & Presentation	Analyzes delivery performance data and generates summary reports alongside Matplotlib visualizations.

4. Proposed Modules

- **Package and Delivery Management:** Focuses on creating a Package base class along with StandardPackage, ExpressPackage, and FragilePackage subclasses. It demonstrates inheritance and polymorphism via methods like calculate_delivery_charges() or get_handling_instructions(), utilizing dictionaries for tracking and sets for delivery zones.
- **Shipment and Delivery Processing:** Records shipments, assigns delivery agents, updates package statuses, and maintains delivery attempt history using lists and

dictionaries. It handles invalid tracking IDs, duplicate shipments, and unavailable agents using robust exception handling.

- **File Management:** Stores customer details, package records, and delivery logs permanently in text files, reading existing records on startup and saving modifications immediately with proper exception handling.
- **Delivery Performance Analysis:** Identifies delayed shipments, high-volume delivery zones, successful delivery rates, and agent performance to generate detailed courier performance reports.
- **Visualization Using Matplotlib:** Uses Matplotlib with actual stored data to generate visual charts for packages by delivery status, monthly shipment volume, and delivery performance by zone.

5. Required Topics from the Subject

Topic	Description & Implementation
Object-Oriented Programming (OOP)	Used to define the Package base class and its subclasses (StandardPackage, ExpressPackage, FragilePackage) to implement inheritance, polymorphism, and modular design.
Lists and Dictionaries	Employed for storing shipment and delivery records, managing agent-wise assignments, and handling fast lookups for tracking IDs.
Exception Handling	Implemented using try-except blocks to gracefully handle invalid tracking IDs, duplicate shipments, unavailable delivery agents, and file I/O errors.
File I/O	Utilized to open, read, write, and permanently

Topic	Description & Implementation
	store customer details, package records, and delivery logs into text files across sessions.

Additional Specifications

Input Specification

The program takes real-world input data from the user and text files, including customer details, package weights and dimensions, destination delivery zones, shipment tracking IDs, delivery agent names, and package status updates.

Output Specification

The program generates console-based confirmation messages, updated text files containing persistent logs, structured tabular delivery performance reports, and Matplotlib visual charts depicting package statuses, monthly shipment volumes, and zone performance.

Constraints and Assumptions

- It is assumed that text files are located in the working directory and have appropriate read/write permissions.
- Tracking IDs are assumed to follow a unique alphanumeric format specified during registration.
- Delivery agents are selected from a pre-configured available pool during shipment processing.

Learning Outcomes

Through this project, students will gain hands-on experience in applying object-oriented programming principles such as inheritance and polymorphism using Python classes. Learners will master file input/output operations to ensure data persistence across program

executions and learn how to handle runtime errors effectively using exception handling mechanisms. Furthermore, students will develop practical skills in processing data structures like lists, tuples, and dictionaries to manage complex business logic, and acquire data visualization capabilities by integrating Matplotlib to generate insightful logistical reports.

Platform Required

Visual Studio Code, Spyder and Github

Books and Link Sources

1. *Python Crash Course* by Eric Matthes.
2. *Automate the Boring Stuff with Python* by Al Sweigart.
3. Official Python Documentation (docs.python.org).
4. Matplotlib Documentation and User Guide (matplotlib.org).